

Rencia App

Guia Universal de Rotas

Mensagens, servidores IPTV, APKs, presença online, backups e integrações

Referência técnica para integração dos aplicativos e do painel

Base principal: <https://enciaapp.manus.space>

Autor: Manus AI Data: 30/08/2026

Sumário

1	Como usar este guia	1
2	Rotas essenciais para qualquer APK	1
2.1	Presença e conteúdo em reprodução	1
2.2	Atualização do conteúdo	1
2.3	Consulta do conteúdo atual	2
2.4	Controles remotos, notificações e failover	2
3	Login, configuração e listas dos APKs	3
3.1	Contrato mínimo recomendado de configuração	3
4	Imagens, mensagens e atualização visual	4
5	Servidores IPTV, mensagens e vencimentos	4
6	Avisos automáticos e Heartbeat do painel	5
7	Backup, importação e Google Drive	5
8	Portal de revenda e loja privada	6
9	Compatibilidade legada e diagnóstico	7
10	Inventário completo extraído do backend	7
10.1	Rotas HTTP	7
10.2	Procedures tRPC	10
11	Checklist de integração do APK	22
12	Referência de segurança e preservação	23
13	Exemplos de referência	24

1 Como usar este guia

Este documento reúne as rotas HTTP públicas e as procedures tRPC atualmente implementadas no Rencia App. Ele foi organizado para que o desenvolvedor de cada APK encontre rapidamente o contrato de que precisa. As rotas antigas foram mantidas no inventário por compatibilidade; não é recomendado trocar uma rota que já funciona sem testar o APK correspondente.

> **Regra principal:** cada aplicativo deve informar seu próprio identificador (app ou app_id) quando o mesmo MAC físico estiver cadastrado em mais de um APK. Isso impede que a atividade do Ouro Pro seja gravada no Future, Excellence, Magnus, Ominus ou em outro aplicativo.

Item	Valor
Base da API	https://enciaapp.manus.space
Formato recomendado	JSON e HTTPS; usar encodeURIComponent nos parâmetros de texto
Identificação	MAC real + ID do aplicativo + session_id único por sessão
Presença	Heartbeat periódico enquanto o APK estiver aberto ou reproduzindo
Isolamento	O backend seleciona o cadastro pelo proprietário, MAC e aplicativo informado

2 Rotas essenciais para qualquer APK

2.1 Presença e conteúdo em reprodução

O APK deve chamar o heartbeat a cada 60 segundos enquanto estiver aberto ou reproduzindo. Quando o canal, filme ou episódio mudar, envie o novo conteúdo imediatamente. O painel conserva o último conteúdo recebido e considera a atividade por até duas horas; o botão **Online agora** usa uma janela curta para mostrar somente os aparelhos ativos naquele momento.

GET https://enciaapp.manus.space/api/v5/heartbeat?mac=AA:BB:CC:DD:EE:FF&app=ouropo&session_id=UUID¤t_content=Nome%20do%20episodio

Chamar a cada 60 segundos durante a reprodução.

Resposta esperada: { "success": true, "mac": "...", "content": "...", "session": { "allowed": true } }

Se o mesmo MAC estiver em mais de um APK, app/app_id é obrigatório.

Se o APK ficar vários minutos no mesmo episódio, ele ainda deve continuar enviando heartbeat. Uma chamada única ao abrir o player não é suficiente. Quando o mesmo MAC estiver em vários aplicativos, app é obrigatório. Respostas HTTP 403 indicam vínculo incorreto e 409 com APP_IDENTIFICATION_REQUIRED indica que o APK precisa informar o aplicativo.

2.2 Atualização do conteúdo

A rota abaixo pode ser usada quando o usuário muda de canal, filme ou episódio. Ela também renova o lastSeen, mas não substitui o heartbeat periódico.

POST <https://enciaapp.manus.space/api/v5/update-watching>

Content-Type: application/json

```
{
  "mac": "AA:BB:CC:DD:EE:FF",
  "app": "ouropro",
  "currentContent": "Nome do canal ou episódio"
}
```

Usar ao trocar o conteúdo. O heartbeat continua obrigatório durante a reprodução.

2.3 Consulta do conteúdo atual

GET <https://enciaapp.manus.space/api/v5/current-watching?mac=AA:BB:CC:DD:EE:FF&app=ouropro>

Resposta:

```
{
  "registered": true,
  "mac": "AA:BB:CC:DD:EE:FF",
  "content": "Nome do canal ou episódio",
  "lastSeen": "2026-08-30T12:00:00.000Z"
}
```

2.4 Controles remotos, notificações e failover

POST <https://enciaapp.manus.space/api/v5/playback-failure>

Content-Type: application/json

```
{ "mac": "AA:BB:CC:DD:EE:FF", "app": "ouropro", "failed_url": "https://servidor.example/
lista" }
```

GET <https://enciaapp.manus.space/api/v5/remote-commands?mac=AA:BB:CC:DD:EE:FF>

POST <https://enciaapp.manus.space/api/v5/remote-commands/ack>

GET <https://enciaapp.manus.space/api/v5/list-notifications?mac=AA:BB:CC:DD:EE:FF>

POST <https://enciaapp.manus.space/api/v5/list-notifications/ack>

Rota	Finalidade
GET /api/v5/remote-commands?mac=...	Consultar um comando remoto pendente
POST /api/v5/remote-commands/ack	Confirmar execução de um comando recebido
GET /api/v5/list-notifications?mac=...	Buscar avisos técnicos confirmados para o MAC
POST /api/v5/list-notifications/ack	Marcar aviso do aplicativo como lido

POST /api/v5/playback-failure	Informar falha de reprodução e solicitar a próxima lista
-------------------------------	--

3 Login, configuração e listas dos APKs

Rota	Finalidade
POST /api/v5/app-login	Login do aplicativo com MAC e credencial configurada
POST /api/v5/login	Login legado do ecossistema v5
POST /api/v5/reseller_login	Login legado de revenda
POST /api/v5/user_register	Registro compatível de usuário do APK
GET /api/v5/apps/:appId/discovery	Descobrir contrato e configuração pública de um aplicativo
GET /api/v5/apps/:appId/config?mac=...	Buscar listas, imagens, mensagens e configuração exclusiva do APK
GET /api/v5/apps/:appId/update?mac=...	Consultar versão e URL de atualização do APK
GET /api/v5/check_mac.php?mac=...	Verificar cadastro, validade e avisos associados ao MAC
GET /api/v5/mac_exists?mac=...	Verificar se o MAC existe
GET /api/v5/get_playlists?mac=...	Obter playlists do dispositivo
GET /api/v5/getdns_list	Obter destinos DNS configurados
GET /api/v5/get_playlist_roku?mac=...	Obter playlist compatível com Roku
GET /api/v5/device_status?mac=...	Consultar situação do dispositivo
GET /api/v5/list_devices	Listagem compatível de dispositivos

3.1 Contrato mínimo recomendado de configuração

Para novos aplicativos, use o identificador técnico do catálogo no parâmetro appId e envie o mesmo identificador no heartbeat. Não reutilize o cadastro de outro APK apenas porque o MAC físico é igual. O painel mantém os registros separados por aplicativo e proprietário.

GET <https://enciaapp.manus.space/api/v5/apps/ouopro/config?mac=AA:BB:CC:DD:EE:FF>

O mesmo padrão pode ser usado para maximus, excellence, future, magnus e ominus, trocando o appId. O servidor devolve somente a configuração autorizada para o aplicativo e o proprietário do cadastro.

4 Imagens, mensagens e atualização visual

Rota	Finalidade
GET /api/v4/logo.php?name=...	Ícone ou logo dinâmico
GET /api/v4/bg.php?name=...	Imagem de fundo dinâmica
GET /api/v4/icon/:name	Ícone visual configurado no painel
GET /api/v5/logo_roku	Logo para Roku
GET /api/v5/bg_roku	Fundo para Roku
GET /api/v5/roku_banners	Banners para Roku
GET /api/config_domain.json	Configuração pública de domínio
GET /config_domain.json	Compatibilidade de configuração de domínio
GET /api/app-config	Configuração geral pública do aplicativo
GET /api/v4/update.php ou /api/update.php	Atualização legada
GET /api/v5/ultra-config?mac=...	Configuração do Ultra/Fusion
GET /api/v5/ultra-update?mac=...	Atualização do Ultra/Fusion
GET /api/v5/maximus-update?mac=...	Atualização do Maximus

As rotas de upload de imagem e APK são administrativas e não devem ser chamadas por APK de cliente. Elas aparecem no inventário completo ao final deste documento para manter a referência de tudo que foi implementado.

5 Servidores IPTV, mensagens e vencimentos

A Central de Servidores IPTV é independente de Usuários. Seus registros não criam MAC, não alteram listas do painel e não misturam aplicativos. O cadastro possui pessoa, telefone, nome/endereço do servidor, vencimento, observação e status Pago/Não pago.

Procedure tRPC	Finalidade
iptvServers.list	Listar somente os servidores da conta autenticada
iptvServers.create	Cadastrar servidor IPTV independente
iptvServers.update	Editar servidor sem tocar em clientes ou MACs
iptvServers.setPaymentStatus	Alternar Pago/Não pago
iptvServers.remove	Excluir somente o registro de servidor autorizado
iptvServers.prepareWhatsApp	Preparar mensagem individual pronta

<code>iptvServers.runAlertsNow</code>	Verificar vencimentos imediatamente
<code>iptvServers.enableDailyAlerts</code>	Ativar verificação diária de avisos
<code>iptvServers.disableDailyAlerts</code>	Parar verificação diária
<code>iptvServers.clearAlertHistory</code>	Apagar histórico com confirmação
<code>iptvServers.prepareWhatsApp</code>	Mantém preparação persistente para futura integração oficial

A mensagem de vencimento deve usar somente o nome da pessoa e a data, sem servidor, lista ou endereço técnico. O envio automático real pelo WhatsApp depende de uma conexão oficial e de modelo aprovado pelo provedor; a mensagem pronta pode abrir o WhatsApp sem envio automático.

6 Avisos automáticos e Heartbeat do painel

Os endpoints abaixo são chamados pelo serviço agendado do painel. Eles não devem ser simulados pelo APK nem substituídos por `setInterval` local.

Endpoint	Finalidade
POST <code>/api/scheduled/automatic-backup</code>	Executar backup automático conforme configuração
POST <code>/api/scheduled/history-retention</code>	Limpar histórico operacional antigo segundo retenção
POST <code>/api/scheduled/list-failover</code>	Verificar falha e recuperação de listas
POST <code>/api/scheduled/iptv-server-alerts</code>	Gerar avisos idempotentes de vencimento de servidores

A automação diária ocorre no backend hospedado. Para os APKs, presença é diferente de agendamento: o APK precisa chamar heartbeat durante a reprodução; o painel não consegue descobrir atividade que nunca foi enviada.

7 Backup, importação e Google Drive

Rota	Finalidade
GET <code>/api/v5/export-backup</code>	Gerar backup completo atual
GET <code>/api/v5/export-backup-v2</code>	Gerar backup compatível com painel legado
POST <code>/api/v5/preview-import-backup</code>	Pré-visualizar importação antes de gravar
POST <code>/api/v5/import-backup</code>	Importar backup após validação

GET /api/backups/:snapshotId/download	Baixar snapshot autorizado
GET /api/google-drive/oauth/callback	Retorno da autorização do Google Drive

O backup completo preserva clientes, MACs, listas, configurações e vínculos conforme o formato exportado. A importação deve ser pré-visualizada e feita somente na conta correta. Não compartilhe tokens, senhas, URLs privadas ou arquivos de backup publicamente.

8 Portal de revenda e loja privada

Rota	Finalidade
POST /api/reseller-portal/login	Login local da revenda
POST /api/reseller-portal/me	Sessão e dados da revenda
POST /api/reseller-portal/dashboard	Resumo do painel da revenda
POST /api/reseller-portal/clients	Listar clientes próprios
POST /api/reseller-portal/client-save	Criar ou editar cliente próprio
POST /api/reseller-portal/client-status	Bloquear ou liberar cliente próprio
POST /api/reseller-portal/client-delete	Excluir cliente próprio
POST /api/reseller-portal/lists	Listar listas próprias
POST /api/reseller-portal/list-save	Criar ou editar lista própria
POST /api/reseller-portal/list-delete	Excluir lista própria
POST /api/reseller-portal/dns	Listar DNS permitido
POST /api/reseller-portal/dns-save	Salvar DNS permitido
POST /api/reseller-portal/dns-delete	Excluir DNS próprio
POST /api/reseller-portal/operations	Consultar operações liberadas

POST /api/reseller-portal/suggestion	Enviar sugestão ao proprietário
GET /api/public/apps	Catálogo público de aplicativos liberados
GET /api/store-invites/:token	Abrir loja por convite

O isolamento por proprietário e revenda é obrigatório: a sessão deve ver e alterar somente os clientes, MACs, listas e aplicativos autorizados para ela. A loja por convite deve mostrar apenas os aplicativos liberados no convite.

9 Compatibilidade legada e diagnóstico

Rota	Finalidade
POST /api/guim.php e GET /api/guim.php	Contrato legado de configuração e atualização
POST /api/v4/guim.php e GET /api/v4/guim.php	Contrato v4 compatível
POST /api/v5/guim.php e GET /api/v5/guim.php	Contrato v5 compatível
POST /api/main.php	Contrato legado principal
GET /player	Compatibilidade de player
GET /player_api.php	Compatibilidade de API do player
GET /api/health	Saúde básica do backend
GET /api/users	Consulta administrativa compatível
GET /api/nuvix/config/:ownerId	Configuração Nuvix por proprietário
GET /apk e GET /ouopro	Rotas de compatibilidade para páginas/APKs
POST /api/chatbot/test	Teste administrativo de chatbot
POST /api/chatbot/revendas	Operação administrativa de chatbot para revendas

10 Inventário completo extraído do backend

A seção seguinte é o inventário bruto das rotas encontradas no código, incluindo aliases mantidos por compatibilidade. Os contratos detalhados e as regras de segurança das rotas sensíveis devem ser conferidos no código do servidor antes de uma integração nova.

10.1 Rotas HTTP

```
GET "/api/oauth/callback", async (req: Request, res: Response) => {
GET "/manus-storage/*", async (req, res) => {
GET "/api/google-drive/oauth/callback", async (req: Request, res: Response) => {
```

```
GET "/api/backups/:snapshotId/download", async (req: Request, res: Response) => {
GET "/api/public/apps", async (_req: Request, res: Response) => {
GET "/api/store-invites/:token", async (req: Request, res: Response) => {
POST "/api/reseller-portal/login", async (req: Request, res: Response) => {
POST "/api/reseller-portal/me", async (req: Request, res: Response) => {
POST "/api/reseller-portal/clients", async (req: Request, res: Response) => {
POST "/api/reseller-portal/dashboard", async (req: Request, res: Response) => {
POST "/api/reseller-portal/client-save", async (req: Request, res: Response) => {
POST "/api/reseller-portal/client-status", async (req: Request, res: Response) => {
POST "/api/reseller-portal/client-delete", async (req: Request, res: Response) => {
POST "/api/reseller-portal/lists", async (req: Request, res: Response) => {
POST "/api/reseller-portal/list-save", async (req: Request, res: Response) => {
POST "/api/reseller-portal/list-delete", async (req: Request, res: Response) => {
POST "/api/reseller-portal/dns", async (req: Request, res: Response) => {
POST "/api/reseller-portal/dns-save", async (req: Request, res: Response) => {
POST "/api/reseller-portal/dns-delete", async (req: Request, res: Response) => {
POST "/api/reseller-portal/operations", async (req: Request, res: Response) => {
POST "/api/reseller-portal/suggestion", async (req: Request, res: Response) => {
GET "/api/users", async (req: Request, res: Response) => {
POST "/api/guim.php", async (req: Request, res: Response) => {
GET "/api/guim.php", async (req: Request, res: Response) => {
POST "/api/v4/guim.php", (req: Request, res: Response, next) => {
GET "/api/v4/guim.php", (req: Request, res: Response, next) => {
POST "/api/v5/guim.php", (req: Request, res: Response, next) => {
GET "/api/v5/guim.php", (req: Request, res: Response, next) => {
POST "/api/v5/check_mac.php", (req: Request, res: Response, next) => {
GET "/api/device/check", async (req: Request, res: Response) => {
POST "/api/upload-image", upload.single("image"), async (req: Request, res: Response) => {
GET "/api/v5/ultra-config", async (req: Request, res: Response) => {
GET "/api/v5/ultra-update", async (req: Request, res: Response) => {
GET "/api/v5/maximus-update", async (req: Request, res: Response) => {
GET "/api/health", (_req: Request, res: Response) => {
GET "/api/v4/logo.php", async (_req: Request, res: Response) => {
GET "/api/v4/bg.php", async (_req: Request, res: Response) => {
GET "/api/app-config", async (_req: Request, res: Response) => {
POST "/api/v5/app-login", async (req: Request, res: Response) => {
GET "/api/v5/apps/:appId/discovery", async (req: Request, res: Response) => {
GET "/api/v5/apps/:appId/config", async (req: Request, res: Response) => {
GET "/api/v5/apps/:appId/update", async (req: Request, res: Response) => {
GET ["/api/v4/update.php", "/api/update.php"], async (_req: Request, res: Response) => {
POST "/api/v4/heartbeat.php", async (req: Request, res: Response) => {
GET "/api/v4/heartbeat.php", async (req: Request, res: Response) => {
POST "/api/chatbot/test", async (req: Request, res: Response) => {
```

```
POST "/api/chatbot/revendas", async (req: Request, res: Response) => {
POST "/api/upload-apk", uploadApk.single("apk"), async (req: Request, res: Response) => {
GET "/apk", async (_req: Request, res: Response) => {
GET "/ouopro", async (_req: Request, res: Response) => {
GET "/api/v4/icon/:name", async (req: Request, res: Response) => {
POST "/api/carousel/upload", uploadCarousel.single("file"), async (req: Request, res:
Response) => {
DELETE "/api/carousel/delete/:id", async (req: Request, res: Response) => {
GET "/api/carousel/list", async (req: Request, res: Response) => {
GET "/api/v5/check_mac.php", async (req: Request, res: Response) => {
GET "/api/v5/mac_exists", async (req: Request, res: Response) => {
GET "/api/v5/check_expire.php", async (req: Request, res: Response) => {
GET "/api/v5/getdns_list", async (_req: Request, res: Response) => {
GET "/api/v5/get_playlist_roku", async (req: Request, res: Response) => {
GET "/api/v5/logo_roku", async (_req: Request, res: Response) => {
GET "/api/v5/bg_roku", async (_req: Request, res: Response) => {
GET "/api/v5/roku_banners", async (_req: Request, res: Response) => {
GET "/api/v5/reseller_contact", async (_req: Request, res: Response) => {
POST "/api/v5/user_register", async (req: Request, res: Response) => {
POST "/api/v5/maximus-test-result", async (req: Request, res: Response) => {
GET "/api/v5/test", async (req: Request, res: Response) => {
POST "/api/v5/login", async (req: Request, res: Response) => {
GET "/api/v5/get_playlists", async (req: Request, res: Response) => {
GET "/api/v5/debug_macs", async (req: Request, res: Response) => {
POST "/api/v5/reseller_login", async (req: Request, res: Response) => {
GET "/api/config_domain.json", handleConfigDomain);
GET "/config_domain.json", async (_req: Request, res: Response) => {
GET "/api/v5/bg_roku", async (req: Request, res: Response) => {
GET "/api/v5/logo_roku", async (req: Request, res: Response) => {
GET "/api/v5/roku_banners", async (req: Request, res: Response) => {
GET "/player", async (_req: Request, res: Response) => {
POST "/api/main.php", async (req: Request, res: Response) => {
GET '/api/nuvix/config/:ownerId', async (req: Request, res: Response) => {
GET '/api/v5/list_devices', async (req: Request, res: Response) => {
GET '/api/v5/device_status', async (req: Request, res: Response) => {
GET '/player_api.php', async (req: Request, res: Response) => {
GET '/api/v5/heartbeat', async (req, res) => {
GET '/api/v5/remote-commands', async (req: Request, res: Response) => {
GET '/api/v5/list-notifications', async (req: Request, res: Response) => {
POST '/api/v5/list-notifications/ack', async (req: Request, res: Response) => {
POST '/api/v5/playback-failure', async (req: Request, res: Response) => {
POST '/api/v5/remote-commands/ack', async (req: Request, res: Response) => {
GET '/api/v5/most-watched', async (req, res) => {
```

```

GET '/api/v5/recently-viewed', async (req, res) => {
GET '/api/v5/current-watching', async (req, res) => {
POST '/api/v5/update-watching', async (req, res) => {
GET '/api/v5/export-backup', async (req: Request, res: Response) => {
GET '/api/v5/export-backup-v2', async (req: Request, res: Response) => {
POST '/api/v5/import-backup', async (req: Request, res: Response) => {
POST '/api/v5/preview-import-backup', async (req: Request, res: Response) => {
POST "/api/scheduled/automatic-backup", async (req: Request, res: Response) => {
GET "/player", (_req: any, res: any) => {
POST "/api/scheduled/history-retention", async (req: Request, res: Response) => {
POST "/api/scheduled/list-failover", async (req: Request, res: Response) => {
POST "/api/scheduled/iptv-server-alerts", async (req: Request, res: Response) => {

```

10.2 Procedures tRPC

```

197:   get: ownerProcedure.query(async () => {
206:       update: ownerProcedure.input(z.object({ origins: z.record(z.string(),
z.string().max(512)) })).mutation(async ({ input }) => {
230:   overview: ownerProcedure.query(async ({ ctx }) => {
238:       googleDriveAuthorizationUrl: ownerProcedure.input(z.object({ origin:
z.string().url() })).mutation(({ ctx, input }) => {
243:   runNow: ownerProcedure.mutation(async ({ ctx }) => {
248:   enableDaily: ownerProcedure.mutation(async ({ ctx }) => {
271:   disableDaily: ownerProcedure.mutation(async ({ ctx }) => {
280:       restore: ownerProcedure.input(z.object({ snapshotId:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
285:       remove: ownerProcedure.input(z.object({ snapshotId:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
291:   clearHistory: ownerProcedure.mutation(async ({ ctx }) => {
300:   overview: protectedProcedure.query(async ({ ctx }) => {
316:   create: protectedProcedure.input(z.object({
345:   update: protectedProcedure.input(z.object({
376:       setPaymentStatus: protectedProcedure.input(z.object({ id:
z.number().int().positive(), paymentStatus: z.enum(["paid", "unpaid"]) })).mutation(async
({ ctx, input }) => {
384:       remove: protectedProcedure.input(z.object({ id:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
393:       prepareWhatsApp: protectedProcedure.input(z.object({ id:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
410:   runAlertsNow: protectedProcedure.mutation(async ({ ctx }) => {
416:   enableDailyAlerts: protectedProcedure.mutation(async ({ ctx }) => {
437:   disableDailyAlerts: protectedProcedure.mutation(async ({ ctx }) => {
447:   clearAlertHistory: protectedProcedure.mutation(async ({ ctx }) => {
455:   prepareWhatsAppBusiness: protectedProcedure.mutation(async ({ ctx }) => {

```

```

469:   retention: ownerProcedure.query(async ({ ctx }) => {
474:   enableRetention: ownerProcedure.mutation(async ({ ctx }) => {
489:   disableRetention: ownerProcedure.mutation(async ({ ctx }) => {
498:       runRetentionNow: ownerProcedure.mutation(async ({ ctx }) =>
cleanupOldOperationalHistory(ctx.user.id)),
499:       listAudit: ownerProcedure.input(z.object({ limit:
z.number().min(1).max(100).optional().default(50) })).query(async ({ ctx, input }) => {
504:       deleteAudit: ownerProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
510:   clearAudit: ownerProcedure.mutation(async ({ ctx }) => {
519:   me: publicProcedure.query(async opts => {
530:   logout: publicProcedure.mutation(({ ctx }) => {
535:   loginLocal: publicProcedure
570:       get: protectedProcedure.input(z.object({ resellerId:
z.number().positive() })).query(async ({ ctx, input }) => {
580:       set: protectedProcedure.input(z.object({ resellerId: z.number().positive(),
permissions: z.array(z.string()).max(RESELLER_PERMISSION_KEYS.length) })).mutation(async
({ ctx, input }) => {
597:   me: protectedProcedure.query(async ({ ctx }) => {
603:       get: protectedProcedure.input(z.object({ resellerId:
z.number().positive() })).query(async ({ ctx, input }) => {
613:       set: protectedProcedure.input(z.object({ resellerId:
z.number().positive(), allowedApps: z.array(z.string()).refine(isManagedAppId, "Aplicativo
inválido.")).max(Object.keys(MANAGED_APP_CATALOG).length) })).mutation(async ({ ctx,
input }) => {
630:   list: ownerProcedure.query(async ({ ctx }) => {
646:   create: ownerProcedure.input(z.object({
668:       revoke: ownerProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
679:       query: protectedProcedure.input(z.object({ term:
z.string().trim().min(2).max(120) })).query(async ({ ctx, input }) => {
703:   overview: ownerProcedure.query(async ({ ctx }) => {
722:   list: ownerProcedure.query(async ({ ctx }) => {
726:       id: resellerBillings.id,
727:       resellerId: resellerBillings.resellerId,
728:       amount: resellerBillings.amount,
729:       status: resellerBillings.status,
730:       dueDate: resellerBillings.dueDate,
731:       paidAt: resellerBillings.paidAt,
732:       recurrenceMonths: resellerBillings.recurrenceMonths,
733:       note: resellerBillings.note,
734:       createdAt: resellerBillings.createdAt,
742:   create: ownerProcedure.input(z.object({

```

```

758:          markPaid:      ownerProcedure.input(z.object({      id:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
774:          remove:      ownerProcedure.input(z.object({      id:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {
787:    list: protectedProcedure
797:    getById: protectedProcedure
805:    linkedApps: protectedProcedure
817:    setLinkedApps: protectedProcedure
843:    recentList: protectedProcedure
853:    stats: protectedProcedure.query(async ({ ctx }) => {
857:    lookupExpiration: protectedProcedure
867:    create: protectedProcedure
917:    update: protectedProcedure
971:    bulkUpdate: protectedProcedure
1004:    delete: protectedProcedure
1022:    deleteMany: protectedProcedure
1029:    deleteExpired: protectedProcedure.mutation(async ({ ctx }) => {
1034:    updateCurrentContent: protectedProcedure
1047:    updateForceShowChannel: protectedProcedure
1060:    bulkUpdateDns: protectedProcedure
1081:    previewBulkSwapDns: protectedProcedure
1092:    bulkSwapDns: protectedProcedure
1140:    expiringSoon: protectedProcedure
1160:    listUniqueUrls: protectedProcedure.query(async ({ ctx }) => {
1182:    overview: protectedProcedure.query(async ({ ctx }) => {
1196:    overview: ownerProcedure.query(async ({ ctx }) => {
1243:    auditLog: ownerProcedure
1254:    diagnostics: ownerProcedure
1284:    list: protectedProcedure.query(async ({ ctx }) => {
1308:    report: protectedProcedure.input(z.object({
1321:    create: protectedProcedure.input(z.object({
1345:    markPaid: protectedProcedure.input(z.object({ id: z.number() })).mutation(async
({ ctx, input }) => {
1356:    attachProof: protectedProcedure.input(z.object({ id: z.number(), proofReference:
z.string().trim().min(1).max(2000) })).mutation(async ({ ctx, input }) => {
1366:    remove: protectedProcedure.input(z.object({ id: z.number() })).mutation(async
({ ctx, input }) => {
1379:    list: ownerProcedure.query(async ({ ctx }) => {
1392:    create: ownerProcedure.input(z.object({
1405:    update: ownerProcedure.input(z.object({
1419:    remove: ownerProcedure.input(z.object({ id: z.number() })).mutation(async ({ ctx,
input }) => {
1426:          applyToExpiration:      ownerProcedure.input(z.object({      id:

```

```

z.number() })).mutation(async ({ ctx, input }) => {
1440:   filters: protectedProcedure.query(async ({ ctx }) => {
1455:     preview: protectedProcedure.input(z.object({
1486:       list:      protectedProcedure.input(z.object({      minutesAgo:
z.number().min(5).max(1440).optional().default(30) })).query(async ({ ctx, input }) => {
1511:       setStatus: protectedProcedure.input(z.object({ id: z.number(), status:
z.enum(["Liberado", "Bloqueado"]) })).mutation(async ({ ctx, input }) => {
1524:     list: ownerProcedure.query(async ({ ctx }) => {
1557:       list:      protectedProcedure.input(z.object({      days:
z.number().min(1).max(90).optional().default(30), status: z.enum(["all", "Liberado",
"Bloqueado", "Expirado"]).optional().default("all") })).query(async ({ ctx, input }) => {
1569:     overview: protectedProcedure.query(async ({ ctx }) => {
1579:     list: protectedProcedure.query(async ({ ctx }) => {
1585:       create: protectedProcedure.input(z.object({ deviceId:
z.number().positive().optional(), title: z.string().trim().min(3).max(255),
description: z.string().max(3000).optional(), priority: z.enum(["low", "medium",
"high", "critical"]).default("medium"), dueAt: z.string().regex(/^\d{4}-\d{2}-
\d{2}$/).optional() })).mutation(async ({ ctx, input }) => {
1595:       update: protectedProcedure.input(z.object({ id: z.number().positive(),
status: z.enum(["open", "in_progress", "resolved", "cancelled"]).optional(),
priority: z.enum(["low", "medium", "high", "critical"]).optional(), assignedToUserId:
z.number().positive().nullable().optional() })).mutation(async ({ ctx, input }) => {
1605:       remove: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1613:     clearFinished: protectedProcedure.mutation(async ({ ctx }) => {
1619:     clearAll: protectedProcedure.mutation(async ({ ctx }) => {
1630:     list: protectedProcedure.query(async ({ ctx }) => {
1645:     get: protectedProcedure.input(z.object({ id: z.number() })).query(async ({ ctx,
input }) => {
1669:     list: protectedProcedure.query(async ({ ctx }) => {
1674:
create: protectedProcedure.input(z.object({ name: z.string().trim().min(1).max(64), color:
z.string().regex(/^[0-9A-Fa-f]{6}$/).default("#D4A72C") })).mutation(async ({ ctx,
input }) => {
1682:       setForDevice: protectedProcedure.input(z.object({ deviceId:
z.number().positive(), tagIds: z.array(z.number().positive()).max(20) })).mutation(async
({ ctx, input }) => {
1696:       create: protectedProcedure.input(z.object({ deviceId: z.number().positive(),
content: z.string().trim().min(1).max(3000) })).mutation(async ({ ctx, input }) => {
1706:       remove: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1714:       remove: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {

```

```

1720:      clearForDevice: protectedProcedure.input(z.object({ deviceId:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1729:    remove: protectedProcedure.input(z.object({ id: z.number().positive(), deviceId:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1735:      clearForDevice: protectedProcedure.input(z.object({ deviceId:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1746:    dnsTargets: protectedProcedure.query(async ({ ctx }) => {
1761:    allTargets: protectedProcedure.query(async ({ ctx }) => {
1767:    list: protectedProcedure.query(async ({ ctx }) => {
1787:    send: protectedProcedure.input(z.object({
1825:    sendToDns: protectedProcedure.input(z.object({
1875:    sendToAll: protectedProcedure.input(z.object({
1896:      cancel: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1903:      delete: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1913:    clearHistory: protectedProcedure.mutation(async ({ ctx }) => {
1925:    list: protectedProcedure.query(async ({ ctx }) => {
1932:      markRead: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1938:    markAllRead: protectedProcedure.mutation(async ({ ctx }) => {
1944:      remove: protectedProcedure.input(z.object({ id:
z.number().positive() })).mutation(async ({ ctx, input }) => {
1950:    clearAll: protectedProcedure.mutation(async ({ ctx }) => {
1959:    policy: protectedProcedure.query(({ ctx }) => {
1977:      clients: protectedProcedure.input(z.object({ search:
z.string().optional() })).query(async ({ ctx, input }) => {
1991:    list: ownerProcedure.query(async ({ ctx }) => {
2018:      check: ownerProcedure.input(z.object({ deviceId: z.number(), deviceId:
z.number().nullable() })).mutation(async ({ ctx, input }) => {
2027:    checkAll: ownerProcedure.mutation(async ({ ctx }) => {
2036:    instabilityReport: ownerProcedure.query(async ({ ctx }) => {
2052:    serverPilot: ownerProcedure.query(async ({ ctx }) => {
2062:    testMacs: ownerProcedure.query(async ({ ctx }) => {
2073:    overview: protectedProcedure.query(async ({ ctx }) => {
2080:    runNow: protectedProcedure.mutation(async ({ ctx }) => {
2088:    enable: protectedProcedure.mutation(async ({ ctx }) => {
2103:    disable: protectedProcedure.mutation(async ({ ctx }) => {
2116:    list: protectedProcedure.query(async ({ ctx }) => {
2142:    create: protectedProcedure.input(z.object({
2236:    update: protectedProcedure.input(z.object({
2258:      remove: protectedProcedure.input(z.object({ id:
z.number().int().positive() })).mutation(async ({ ctx, input }) => {

```

2272: list: protectedProcedure
2281: copyTargets: protectedProcedure
2291: add: protectedProcedure
2318: update: protectedProcedure
2348: duplicateToDevices: protectedProcedure
2366: delete: protectedProcedure
2386: list: protectedProcedure
2396: stats: protectedProcedure.query(async ({ ctx }) => {
2400: create: protectedProcedure
2455: update: protectedProcedure
2498: toggleBlock: protectedProcedure
2530: delete: protectedProcedure
2562: list: protectedProcedure.query(async () => {
2570: info: protectedProcedure.query(async ({ ctx }) => {
2589: getPublic: publicProcedure.query(async () => {
2600: getAll: protectedProcedure.query(async ({ ctx }) => {
2614: update: protectedProcedure
2630: updateMany: protectedProcedure
2645: uploadImage: protectedProcedure
2667: getUploadUrl: protectedProcedure
2695: getSettings: protectedProcedure.query(async ({ ctx }) => {
2715: updateSettings: protectedProcedure
2757: list: adminProcedure
2784: stats: adminProcedure.query(async () => {
2796: updateRole: adminProcedure
2806: profile: protectedProcedure.query(({ ctx }) => ctx.user),
2807: updateProfile: protectedProcedure
2817: changeCredentials: protectedProcedure
2836: list: protectedProcedure.query(async ({ ctx }) => {
2841: create: protectedProcedure
2861: update: protectedProcedure
2877: delete: protectedProcedure
2885: applyGroupToDevices: protectedProcedure
2906: groupHealth: protectedProcedure.query(async ({ ctx }) => {
2924: listServerBlocks: protectedProcedure.query(async ({ ctx }) => {
2930: setServerBlock: protectedProcedure
2942: createMaintenanceNotice: ownerProcedure
2961: list: protectedProcedure
2973: slides: publicProcedure.query(async () => {
2982: config: publicProcedure.query(async () => {
2990: adminList: adminProcedure.query(async () => {
2997: createSlide: adminProcedure
3013: updateSlide: adminProcedure

```

3032:    deleteSlide: adminProcedure
3042:    updateConfig: adminProcedure
3064:    create: protectedProcedure
3081:    list: protectedProcedure.query(async ({ ctx }) => {
3093:    list: protectedProcedure.query(async ({ ctx }) => {
3114:    create: adminProcedure
3132:    delete: adminProcedure
3144:    create: adminProcedure
3180:    update: adminProcedure
3210:    delete: adminProcedure
3222:    getConfig: publicProcedure
3232:    updateConfig: protectedProcedure
3273:    updateCurrentContent: publicProcedure
3297:    appStats: protectedProcedure.query(async ({ ctx }) => {
=== HEARTBEAT/WEBHOOK/SCHEDULE ===
server/_core/heartbeat.ts:12:  /** Callback path. MUST start with `/api/scheduled/`. */
server/_core/heartbeat.ts:128:  if (!path || !path.startsWith("/api/scheduled/")) {
server/_core/heartbeat.ts:131:      message: "callback path must start with /api/
scheduled/",
server/_core/heartbeat.ts:199: * owner-only via the sandbox CLI (`manus-heartbeat list --
user-id <uid>`).
server/_core/notification.ts:66:export async function notifyOwner(
server/_core/notification.ts:102:      `[Notification] Failed to notify owner
(${response.status} ${response.statusText})${
server/_core/systemRouter.ts:2:import { notifyOwner } from "./notification";
server/_core/systemRouter.ts:16:  notifyOwner: adminProcedure
server/_core/systemRouter.ts:24:      const delivered = await notifyOwner(input);
server/apiRoutes.ts:33:import { normalizeHeartbeatContent, readHeartbeatContent } from "./
heartbeatContent";
server/apiRoutes.ts:48:import { buildHeartbeatMacLookup } from "./heartbeatMac";
server/apiRoutes.ts:294:  const whatsappRaw = (cfg.contact_whatsapp || "").replace(/[^\d+]/
g, "");
server/apiRoutes.ts:295:  const whatsappNumber = whatsappRaw.replace(/D/g, "");
server/apiRoutes.ts:296:      const whatsappUrl = whatsappNumber ? `https://wa.me/
${whatsappNumber}` : "";
server/apiRoutes.ts:307:  if (!lockButtonUrl) lockButtonUrl = whatsappUrl;
server/apiRoutes.ts:315:  // Formatar str_whatsapp como URL do WhatsApp para o APK abrir
server/apiRoutes.ts:316:  const strWhatsapp = whatsappUrl || whatsappNumber;
server/apiRoutes.ts:328:    str_whatsapp: strWhatsapp,
server/apiRoutes.ts:1145:      str_whatsapp: words.str_whatsapp,
server/apiRoutes.ts:1152:      contact_whatsapp: words.str_whatsapp,
server/apiRoutes.ts:1691:    * Resposta: { background_url, logo_url, banner_url,
support_text, contact_whatsapp }

```

```

server/apiRoutes.ts:1701:         contact_whatsapp: cfg.contact_whatsapp || "",
server/apiRoutes.ts:1954:     * POST /api/v4/heartbeat.php
server/apiRoutes.ts:1956:     * Endpoint de heartbeat para o APK reportar o conteúdo assistido
em tempo real.
server/apiRoutes.ts:1964:     app.post("/api/v4/heartbeat.php", async (req: Request, res:
Response) => {
server/apiRoutes.ts:2086:         console.error("[API] /api/v4/heartbeat.php error:", error);
server/apiRoutes.ts:2091:     // GET /api/v4/heartbeat.php – retorna o conteúdo atual de
um dispositivo
server/apiRoutes.ts:2092:     app.get("/api/v4/heartbeat.php", async (req: Request, res:
Response) => {
server/apiRoutes.ts:2116:         console.error("[API] GET /api/v4/heartbeat.php error:",
error);
server/apiRoutes.ts:2666:         const gpcWhatsapp = (cfg.gpcpro_contact_whatsapp ||
cfg.contact_whatsapp || "").trim();
server/apiRoutes.ts:2668:         const gpcResellerWhatsapp = (cfg.gpcpro_reseller_whatsapp
|| cfg.reseller_whatsapp || cfg.contact_whatsapp || "").trim();
server/apiRoutes.ts:2692:         whatsapp_url: gpcWhatsapp.replace(/\\D/g, "") ? `https://
wa.me/${gpcWhatsapp.replace(/\\D/g, "")}` : "",
server/apiRoutes.ts:2694:         reseller_whatsapp: gpcResellerWhatsapp,
server/apiRoutes.ts:3093:         const whatsappRaw = (cfg.gpcpro_contact_whatsapp ||
cfg.contact_whatsapp || "").replace(/[\\^d+]/g, "");
server/apiRoutes.ts:3094:         const whatsappNumber = whatsappRaw.replace(/\\D/g, "");
server/apiRoutes.ts:3098:         whatsapp: whatsappNumber ? `https://wa.me/${whatsappNumber}
` : "",
server/apiRoutes.ts:3099:         whatsapp_number: whatsappNumber,
server/apiRoutes.ts:3101:         phone: whatsappNumber,
server/apiRoutes.ts:3110:     res.status(500).json({ name: "", whatsapp: "", website: "" });
server/apiRoutes.ts:4502: // Endpoint para registrar heartbeat do dispositivo
server/apiRoutes.ts:4503: app.get('/api/v5/heartbeat', async (req, res) => {
server/apiRoutes.ts:4575:     console.error('[API] /api/v5/heartbeat error:', error);
server/apiRoutes.ts:4576:     res.json({ success: false, message: 'Erro ao registrar
heartbeat' });
server/apiRoutes.ts:4582:     * 0 APK consulta esta rota no mesmo ciclo do heartbeat. Um
único comando é entregue por vez.
server/apiRoutes.ts:4804:     * GET /api/v5/current-watching?mac=XX:XX:XX:XX:XX:XX
server/apiRoutes.ts:4807: app.get('/api/v5/current-watching', async (req, res) => {
server/apiRoutes.ts:4854:         console.error('[API] /api/v5/current-watching error:',
error);
server/apiRoutes.ts:4860:     * POST /api/v5/update-watching
server/apiRoutes.ts:4864: app.post('/api/v5/update-watching', async (req, res) => {
server/apiRoutes.ts:4913:         console.error('[API] /api/v5/update-watching error:',
error);

```

```

server/backupSchedule.ts:9:    app.post("/api/scheduled/automatic-backup", async (req:
Request, res: Response) => {
server/heartbeatContent.test.ts:2:import          {          normalizeHeartbeatContent,
readHeartbeatContent } from "./heartbeatContent";
server/heartbeatContent.test.ts:4:describe("heartbeat de conteúdo assistido", () => {
server/heartbeatContent.test.ts:5:  it("mantém o último conteúdo quando o APK envia apenas
um heartbeat vazio", () => {
server/heartbeatContent.ts:3: * Um heartbeat vazio apenas confirma que o aparelho permanece
conectado.
server/historyRetentionSchedule.ts:9:  app.post("/api/scheduled/history-retention", async
(req: Request, res: Response) => {
server/listFailoverSchedule.ts:9:  app.post("/api/scheduled/list-failover", async (req:
Request, res: Response) => {
server/resellerReport.test.ts:14:  it("separa cadastro, ativação confirmada por heartbeat
e clientes online", () => {
server/resellerReport.ts:10:/** Métricas baseadas em cadastro e heartbeat real, sem
contabilizar clique como download. */
server/routers.ts:42:import { createHeartbeatJob, deleteHeartbeatJob } from "./_core/
heartbeat";
server/routers.ts:261:      path: "/api/scheduled/automatic-backup",
server/routers.ts:302:      if (!db) return { servers: [], expiring: [], alerts: [],
setting: null, whatsappBusiness: { status: "not_configured", enabled: false } };
server/routers.ts:307:      const whatsappBusiness = (await db.select().from(iptvServerWhatsAppBusinessS
ctx.user.id)).limit(1))[0] ?? { status: "not_configured" as const, enabled: false };
server/routers.ts:314:      return { servers, expiring, alerts, setting, whatsappBusiness };
server/routers.ts:405:                                eq(iptvServerAlertLogs.serverId,
server.id), eq(iptvServerAlertLogs.alertDate, alertDate), eq(iptvServerAlertLogs.channel,
"whatsapp_ready"),
server/routers.ts:407:                                if      (!existing)      await
db.insert(iptvServerAlertLogs).values({  serverId:  server.id,  ownerId:  ctx.user.id,
alertDate, channel: "whatsapp_ready", message });
server/routers.ts:430:      path: "/api/scheduled/iptv-server-alerts",
server/routers.ts:484:      const job = await createHeartbeatJob({ name: `limpeza-
historicos-${ctx.user.id}`, cron: HISTORY_RETENTION_CRON, path: "/api/scheduled/history-
retention", description: "Limpeza diária de históricos operacionais com retenção de 3
dias" }, sessionToken);
server/routers.ts:2098:      const job = await createHeartbeatJob({ name: `monitor-
listas-${ctx.user.id}`, cron: LIST_FAILOVER_CRON, path: "/api/scheduled/list-failover",
description: "Monitoramento e troca automática de listas a cada 10 minutos" },
sessionToken);
server/xstreamcodes.test.ts:6: * Valida endpoints: /player_api.php, /api/v5/heartbeat
server/xstreamcodes.test.ts:90:  describe('GET /api/v5/heartbeat - Device Heartbeat', ()
=> {

```

```

server/xstreamcodes.test.ts:136:          // Este teste valida que o heartbeat atualiza o
timestamp
server/xstreamcodes.test.ts:139:          // Simular heartbeat
server/xstreamcodes.test.ts:168:      it('device deve aparecer como online após heartbeat',
async () => {
server/appSessionControl.test.ts:23:      it("mantém autorizada a sessão que apenas renovou o
heartbeat", () => {
server/appSessionControl.ts:5:/** APKs devem renovar a sessão pelo heartbeat antes deste
período terminar. */
server/appServerDirectory.test.ts:16:          heartbeat_url: "https://appplus.top/api/v5/
heartbeat",
server/appServerDirectory.ts:36:      heartbeat_url: `${origin}/api/v5/heartbeat`,
server/iptvServerAlertSchedule.ts:9:      app.post("/api/scheduled/iptv-server-alerts", async
(req: Request, res: Response) => {
server/heartbeatMac.test.ts:2:import { buildHeartbeatMacLookup } from "../heartbeatMac";
server/heartbeatMac.test.ts:4:describe("heartbeat MAC lookup", () => {
client/src/components/AIChatBox.tsx:25:      * Callback when user sends a message.
client/src/pages/ComponentShowcase.tsx:1413:          placeholder="Try sending
a message..."
client/src/pages/ListMonitor.tsx:47:          {macTests.data    &&    <Card><CardContent
className="flex flex-wrap items-center justify-between gap-3 p-4"><div><p className="flex
items-center gap-2 font-semibold"><Wifi size={17} className="text-primary" /> Teste
em massa de MACs</p><p className="mt-1 text-sm text-muted-foreground">Usa o
último heartbeat de cada aplicativo para indicar conexão nos últimos
15 minutos.</p></div><div className="flex gap-4 text-sm"><span className="font-
medium text-emerald-600">{macTests.data.online} online</span><span className="font-
medium text-rose-600">{macTests.data.offline} offline</span><span className="text-muted-
foreground">{macTests.data.checked} testados</span></div></CardContent></Card>
client/src/pages/MaximusPlayer.tsx:82:          const response = await fetch('/api/v4/
heartbeat.php', {
client/src/pages/MaximusPlayer.tsx:91:          console.log('Canal atualizado no heartbeat:',
data);
client/src/pages/RemoteCommands.tsx:120:          <CardHeader><div className="flex flex-
col gap-3 sm:flex-row sm:items-start sm:justify-between"><div><CardTitle className="flex
items-center gap-2 text-base"><MonitorCog size={18} /> Histórico e confirmação</
CardTitle><CardDescription className="mt-1">0 APK consulta os comandos junto do heartbeat
e confirma a execução. Não há confirmação falsa: a ordem fica pendente até o aparelho
responder.</CardDescription></div><Button variant="outline" size="sm" className="w-fit
gap-2 border-destructive/40 text-destructive hover:bg-destructive hover:text-destructive-
foreground" disabled={!completedRows.length || clearHistory.isPending} onClick={() => { if
(window.confirm(`Excluir ${completedRows.length} comando(s) finalizado(s) do histórico?
`)) clearHistory.mutate(); }}><Trash2 size={15} /> Limpar histórico</Button></div></
CardHeader>

```

```

client/src/pages/RemoteCommands.tsx:162:      sendToDns.mutate({ dnsHost, command,
payload: buildPayload(command, listIndex, dns, message), expiresInMinutes:
Number(expiresInMinutes) });
client/src/pages/RemoteCommands.tsx:199:      sendToAll.mutate({ command,
payload: buildPayload(command, listIndex, dns, message), expiresInMinutes:
Number(expiresInMinutes) });
client/src/pages/ResellerReport.tsx:17:      <div className="flex flex-col gap-3
sm:flex-row sm:items-end sm:justify-between"><div><div className="mb-1 flex items-center
gap-2 text-xs font-bold uppercase tracking-[.18em] text-primary"><BarChart3 size={17} /
> Visão do proprietário</div><h1 className="text-2xl font-bold">Relatório de Revendas</
h1><p className="mt-1 text-sm text-muted-foreground">Controle de clientes, APKs ativados
por heartbeat e desempenho de cada revenda.</p></div><div className="flex gap-2"><Button
variant="outline" className="gap-2" onClick={exportReport}><Download size={16} /> CSV</
Button><Button className="gap-2 text-black dark:text-white" disabled={query.isFetching}
onClick={() => query.refetch()}><RefreshCw size={16} className={query.isFetching ?
"animate-spin" : ""} /> Atualizar</Button></div></div>
client/src/pages/ResellerReport.tsx:19:      <Card><CardHeader><CardTitle className="flex
items-center gap-2 text-base"><Trophy size={17} /> Ranking e desempenho por
revenda</CardTitle><CardDescription>"APK ativado" significa que o dispositivo
cadastrado enviou pelo menos um heartbeat. Não conta clique como download.</
CardDescription></CardHeader><CardContent className="p-0"><div className="overflow-x-
auto"><table className="w-full min-w-[1120px] text-sm"><thead className="border-y bg-
muted/40 text-left text-xs uppercase tracking-wide text-muted-foreground"><tr><th
className="px-5 py-3">#</th><th className="px-5 py-3">Revenda</th><th className="px-5
py-3">Clientes</th><th className="px-5 py-3">APKs</th><th className="px-5 py-3">Apps</
th><th className="px-5 py-3">Limite</th><th className="px-5 py-3">Vencimentos</
th><th className="px-5 py-3">Financeiro</th><th className="px-5 py-3">Status</th></
tr></thead><tbody>{ranking.map((row, index) => <tr key={row.id} className="border-
b last:border-0"><td className="px-5 py-4 font-semibold text-primary">{index + 1}
</td><td className="px-5 py-4"><p className="font-medium">{row.name || "Sem nome"}</
p><p className="text-xs text-muted-foreground">{row.email || "Sem e-mail"} · Validade:
{date(row.planValidade)}</p></td><td className="px-5 py-4"><strong>{row.clientCount}
</strong><span className="ml-1 text-xs text-muted-foreground">({row.activeClients}
liberados)</span></td><td className="px-5 py-4"><strong>{row.apkActivatedClients}</
strong><p className="text-xs text-muted-foreground">{row.onlineClients} online
agora</p></td><td className="px-5 py-4"><div className="flex max-w-64 flex-
wrap gap-1">{row.appBreakdown.length ? row.appBreakdown.map((item) => <Badge
key={item.app} variant="secondary">{item.app}: {item.activated}/{item.clients}</Badge>) :
<span className="text-xs text-muted-foreground">Sem clientes</span></div></td><td
className="px-5 py-4">{row.clientCount} / {row.limiteDevices ?? 0}<p className="text-
xs text-muted-foreground">Restam {row.remainingDevices}</p></td><td className="px-5
py-4"><Badge variant={row.expiringSoon ? "destructive" : "secondary"}
>{row.expiringSoon} próximos</Badge></td><td className="px-5 py-4"><p className="font-

```

```

medium text-emerald-600">{money(row.finance.received)}</p><p className="text-xs text-
muted-foreground">Pendente: {money(row.finance.pending)}</p></td><td className="px-5
py-4"><Badge variant={row.isActive ? "default" : "destructive"}>{row.isActive ? "Ativa" :
"Bloqueada"}</Badge></td></tr>)}{!query.isLoading && rows.length === 0 && <tr><td
colSpan={9} className="px-5 py-14 text-center text-muted-foreground">Nenhuma revenda
cadastrada.</td></tr></tbody></table></div></CardContent></Card>
client/src/pages/Settings.tsx:22: contact_whatsapp: "",
client/src/pages/Settings.tsx:418: value={form.contact_whatsapp}
client/src/pages/Settings.tsx:419: onChange={e =>
handleChange("contact_whatsapp", e.target.value)}
client/src/pages/Settings.tsx:422: <p className="text-xs text-
muted-foreground">Usado no botão de WhatsApp do app (campo <code>str_whatsapp</code>).</p>
client/src/pages/SettingsGpcpro.tsx:20: gpcpro_contact_whatsapp: "",
client/src/pages/SettingsGpcpro.tsx:37: gpcpro_reseller_whatsapp: "",
client/src/pages/SettingsGpcpro.tsx:271: value={form.gpcpro_contact_whatsapp}
client/src/pages/SettingsGpcpro.tsx:272: onChange={e =>
handleChange("gpcpro_contact_whatsapp", e.target.value)}
client/src/pages/
SettingsGpcpro.tsx:318: value={form.gpcpro_reseller_whatsapp}
client/src/pages/SettingsGpcpro.tsx:319: onChange={e =>
handleChange("gpcpro_reseller_whatsapp", e.target.value)}
client/src/pages/IptvServers.tsx:144: <Card className="border-
primary/20"><CardHeader className="gap-3 sm:flex-row sm:items-center sm:justify-
between"><div><CardTitle className="flex items-center gap-2 text-base"><CalendarClock /
> Avisos automáticos</CardTitle><CardDescription>0 painel verifica os vencimentos
diariamente e cria um alerta uma vez por dia para cada servidor.</CardDescription></
div><div className="flex flex-wrap gap-2">{hasDailySchedule ? <Button variant="outline"
size="sm" onClick={() => disableAlerts.mutate()} disabled={disableAlerts.isPending}
className="gap-1.5"><Power size={15} /> Pausar</Button> : <Button size="sm" onClick={()
=> enableAlerts.mutate()} disabled={enableAlerts.isPending} className="gap-1.5 text-
black"><CheckCircle2 size={15} /> Ativar avisos</Button>}</div></CardHeader><CardContent
className="space-y-3"><p className="rounded-lg bg-muted/50 p-3 text-xs text-muted-
foreground">A mensagem do WhatsApp já fica pronta para abrir e enviar. 0
envio automático por WhatsApp Business poderá ser ativado depois, sem recriar
os servidores.</p><div className="flex flex-col gap-3 rounded-lg border border-
dashed p-3 sm:flex-row sm:items-center sm:justify-between"><div><p className="text-
sm font-semibold">WhatsApp Business</p><p className="text-xs text-muted-foreground">A
conexão oficial será configurada depois com o número e a chave da conta
Business; seus servidores atuais serão aproveitados.</p></div><div className="flex
items-center gap-2"><Badge variant={overview.data?.whatsappBusiness.status ===
"active" ? "default" : overview.data?.whatsappBusiness.status ===
"ready" ? "secondary" : "outline"}>{overview.data?.whatsappBusiness.status ===
"active" ? "Ativo" : overview.data?.whatsappBusiness.status === "ready" ?

```

```
"Preparado" : "Aguardando conexão"</Badge>{overview.data?.whatsappBusiness.status
=== "not_configured" && <Button variant="outline" size="sm" onClick={() =>
prepareWhatsAppBusiness.mutate()} disabled={prepareWhatsAppBusiness.isPending}>Preparar</
Button>}</div></div></CardContent></Card>
```

```
client/src/pages/IptvServers.tsx:152: <Card><CardHeader className="gap-3 sm:flex-
row sm:items-start sm:justify-between"><div><CardTitle className="text-base">Histórico
de avisos</CardTitle><CardDescription>Registros gerados pela verificação automática e
pela abertura de mensagens prontas.</CardDescription></div><Button variant="outline"
size="sm" className="w-fit text-destructive hover:text-destructive" disabled={!
overview.data?.alerts.length || clearAlertHistory.isPending} onClick={() => { if
(window.confirm("Apagar todas as mensagens do histórico de servidores? Isso não apaga
servidores, clientes, MACs ou listas.)) clearAlertHistory.mutate(); }}><Trash2 size={15}
className="mr-1.5" /> Apagar mensagens</Button></CardHeader><CardContent className="space-
y-2">{overview.data?.alerts.slice(0, 8).map((alert) => <div key={alert.id} className="flex
gap-3 rounded-lg border p-3"><Clock3 className="mt-0.5 shrink-0 text-muted-
foreground" size={16} /><div className="min-w-0"><p className="text-sm">{alert.message}
</p><p className="mt-1 text-xs text-muted-foreground">{alert.channel === "panel" ?
"Aviso no painel" : alert.channel === "whatsapp_ready" ? "Mensagem pronta para
WhatsApp" : "WhatsApp Business"} · {new Date(alert.createdAt).toLocaleString("pt-BR")}</
p></div></div>)}{!overview.data?.alerts.length && <p className="py-4 text-sm text-muted-
foreground">0s avisos aparecerão aqui depois da primeira verificação.</p>}</CardContent></
Card>
```

11 Checklist de integração do APK

Nº	Verificação
1	Usar HTTPS e o domínio atual configurado para o painel.
2	Enviar MAC real, normalizado pelo próprio servidor, sem trocar o MAC entre aplicativos.
3	Enviar app ou app_id correto em cada heartbeat quando houver cadastros duplicados por aplicativo.
4	Gerar session_id UUID ao iniciar uma sessão e reutilizá-lo até fechar o APK.
5	Chamar heartbeat a cada 60 segundos durante a reprodução.
6	Enviar current_content ao iniciar e ao trocar canal, filme ou episódio.
7	Tratar 403, 409 e CONNECTION_LIMIT_REACHED sem esconder silenciosamente o erro.
8	Não apagar o último conteúdo quando o heartbeat vier sem texto; vazio significa que não há atualização de título.
9	Testar o APK em celular e TV Box, porque o tipo de dispositivo pode alterar o caminho visual.
10	Não enviar credenciais, tokens ou URLs privadas em logs públicos.

12 Referência de segurança e preservação

As rotas administrativas usam autenticação e isolamento por proprietário. As rotas públicas de APK recebem apenas os dados necessários para o funcionamento do aplicativo. A existência de uma rota pública não significa que ela permita editar o painel. Qualquer alteração de banco, cliente, MAC, lista, validade, credencial ou configuração deve ser feita somente por uma operação autorizada e validada.

Este guia foi gerado a partir do inventário atual do projeto em 30/08/2026. Rotas legadas permanecem documentadas para facilitar manutenção, mas a integração nova deve preferir os contratos v5 e o identificador explícito do aplicativo.

13 Exemplos de referência

Códigos importantes para o APK:

200 – chamada aceita; ler success/registered e continuar o fluxo.

400 – parâmetro obrigatório ausente ou inválido; revisar MAC e campos enviados.

403 – o MAC não está vinculado ao aplicativo informado; não tentar usar cadastro de outro APK.

404 – dispositivo ou rota de recurso não encontrado; mostrar estado de cadastro conforme a tela.

409 APP_IDENTIFICATION_REQUIRED – o MAC existe em vários aplicativos; enviar app/app_id.

409 CONNECTION_LIMIT_REACHED – limite de sessões atingido; interromper a reprodução conforme a regra do painel.

503 – serviço temporariamente indisponível; tentar novamente com espera progressiva.

Nunca registrar em log público tokens, senhas, URLs privadas ou o conteúdo completo de respostas administrativas.